1. Which core Python scientific library provides advanced signal processing, numerical optimization, and integration submodules?

(A) NumPy
(B) SciPy
(C) SymPy
(D) Pandas

2. According to the Nyquist-Shannon sampling theorem, what is the minimum sampling frequency f_s required to avoid aliasing for a signal with maximum frequency f_{max}?

(A) $f_s \ge f_{max}$
(B) $f_s \ge 2 f_{max}$
(C) $f_s \ge 4 f_{max}$
(D) $f_s \ge \pi f_{max}$

3. What phenomenon occurs when a continuous signal contains frequency components higher than half the sampling rate ($f_s / 2$)?

(A) Phase jitter
(B) Aliasing distortion
(C) Harmonic resonance
(D) Thermal drift

4. What is the frequency $f_s / 2$ universally called in digital signal processing?

(A) Resonant frequency
(B) Nyquist frequency
(C) Carrier frequency
(D) Corner frequency

5. How is the Signal-to-Noise Ratio (SNR) in decibels (dB) mathematically defined in terms of signal power P_s and noise power P_n?

(A) $\text{SNR} = 10 \log_{10}(P_s / P_n)$
(B) $\text{SNR} = 20 \log_{10}(P_s / P_n)$
(C) $\text{SNR} = P_s / P_n$
(D) $\text{SNR} = \log_e(P_s - P_n)$

6. Which module is the industry standard for serial port (RS-232, UART, USB CDC) communication in Python?

(A) serialport
(B) pySerial (serial)
(C) pylink
(D) uartsys

7. What is a standard default baud rate for microcontroller UART serial debugging in engineering labs?

(A) 1200
(B) 9600 or 115200
(C) 1000000
(D) 48000

8. How do you open a serial port on Windows at COM3 with baud rate 115200 in Python?

(A) `ser = serial.Serial('COM3', 115200)`
(B) `ser = serial.open('COM3', 115200)`
(C) `ser = serial.Port(3, 115200)`
(D) `ser = serial.connect('COM3')`

9. What parameter in `serial.Serial` specifies the maximum wait time for incoming bytes before unblocking?

(A) wait_ms
(B) timeout
(C) delay
(D) block_time

10. Why must string commands sent over a serial port be encoded via `.encode('utf-8')`?

(A) Serial ports transmit raw binary bytes, not Python unicode string objects
(B) For data encryption
(C) To compress commands
(D) To set baud rate

11. Which built-in module provides low-level network socket interface programming in Python?

(A) network
(B) socket
(C) http
(D) urllib

12. What address family constant represents standard IPv4 network addressing in `socket.socket()`?

(A) socket.AF_INET6
(B) socket.AF_INET
(C) socket.AF_UNIX
(D) socket.AF_IP4

13. What socket type represents connection-oriented, reliable TCP byte streams?

(A) socket.SOCK_DGRAM
(B) socket.SOCK_STREAM
(C) socket.SOCK_RAW
(D) socket.SOCK_RDM

14. What socket type represents connectionless, lightweight UDP datagrams?

(A) socket.SOCK_STREAM
(B) socket.SOCK_DGRAM
(C) socket.SOCK_SEQPACKET
(D) socket.SOCK_RAW

15. Which method initiates a TCP connection from a client socket to a remote server host and port?

(A) sock.bind()
(B) sock.connect()
(C) sock.listen()
(D) sock.link()

16. Which method associates a server socket with a specific network interface IP and port number?

(A) sock.attach()
(B) sock.bind()
(C) sock.assign()
(D) sock.host()

17. Which method sets a TCP server socket into passive listening mode to accept client connections?

(A) sock.wait()
(B) sock.listen()
(C) sock.serve()
(D) sock.ready()

18. What does `client_socket, address = server_socket.accept()` return?

(A) A new socket dedicated to communication with the connected client, and client IP/port address tuple
(B) Boolean success flag
(C) Raw byte stream
(D) Port number

19. Which method receives incoming bytes from a connected TCP socket?

(A) sock.read()
(B) sock.recv(buffer_size)
(C) sock.get_bytes()
(D) sock.fetch()

20. What is the standard simulation time array for a 1-second signal with 10 kHz sampling rate in NumPy?

(A) `t = np.arange(0, 1.0, 1/10000)`
(B) `t = np.linspace(0, 10000, 1)`
(C) `t = range(10000)`
(D) `t = np.zeros(1000)`

21. Which mathematical function generates a continuous sine wave of frequency f and phase ϕ?

(A) `A * np.sin(2 * np.pi * f * t + phi)`
(B) `A * np.sin(f * t)`
(C) `np.cos(2 * f * t)`
(D) `A * np.sin(2 * f * t)`

22. Which function from `scipy.signal` generates a periodic square wave with a specified duty cycle?

(A) scipy.signal.pulse()
(B) scipy.signal.square()
(C) scipy.signal.rect()
(D) scipy.signal.step()

23. Which function from `scipy.signal` generates a periodic sawtooth or triangle wave?

(A) scipy.signal.triangle()
(B) scipy.signal.sawtooth()
(C) scipy.signal.ramp()
(D) scipy.signal.slope()

24. Which function from `scipy.signal` generates a swept-frequency cosine (chirp) signal?

(A) scipy.signal.sweep()
(B) scipy.signal.chirp()
(C) scipy.signal.vco()
(D) scipy.signal.modulate()

25. What is an impulse response $h[n]$ of a Linear Time-Invariant (LTI) system?

(A) The system output when excited by a unit Dirac delta impulse $\delta[n]$
(B) The step response derivative
(C) The frequency spectrum
(D) The transfer function poles

26. What mathematical operation relates LTI system input $x[n]$, impulse response $h[n]$, and output $y[n]$?

(A) Multiplication
(B) Convolution $y[n] = x[n] * h[n]$
(C) Correlation
(D) Differentiation

27. Which function from `scipy.io.wavfile` reads uncompressed audio WAV files into a sampling rate and NumPy array?

(A) scipy.io.wavfile.load()
(B) scipy.io.wavfile.read()
(C) scipy.io.wavfile.open()
(D) scipy.io.wavfile.parse()

28. Which function from `scipy.io.wavfile` saves a NumPy array as an audio WAV file on disk?

(A) scipy.io.wavfile.save()
(B) scipy.io.wavfile.write()
(C) scipy.io.wavfile.export()
(D) scipy.io.wavfile.dump()

29. What is the simplest numerical algorithm for solving initial value ordinary differential equations (ODEs)?

(A) Runge-Kutta 4th order
(B) Euler's Forward Method
(C) Adams-Bashforth
(D) Crank-Nicolson

30. In Euler's forward method, how is the state at time step $n+1$ updated from state y_n and derivative $f(t_n, y_n)$?

(A) $y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n)$
(B) $y_{n+1} = y_n + \frac{\Delta t}{2} f(t_n, y_n)$
(C) $y_{n+1} = y_n - \Delta t \cdot f(t_n, y_n)$
(D) $y_{n+1} = f(t_n, y_n) / \Delta t$

31. What is the global truncation error order of Euler's forward method?

(A) $\mathcal{O}(\Delta t^2)$
(B) $\mathcal{O}(\Delta t)$
(C) $\mathcal{O}(\Delta t^4)$
(D) $\mathcal{O}(1)$

32. Which modern SciPy function is the recommended high-level solver for initial value ordinary differential equations?

(A) scipy.integrate.odeint()
(B) scipy.integrate.solve_ivp()
(C) scipy.integrate.quad()
(D) scipy.integrate.ode()

33. What is the default integration method employed by `scipy.integrate.solve_ivp`?

(A) 'Euler'
(B) 'RK45' (Explicit Runge-Kutta of order 5(4))
(C) 'BDF'
(D) 'Radau'

34. Which lightweight publish-subscribe messaging protocol is ubiquitous in IoT sensor telemetry?

(A) HTTP/1.1
(B) MQTT (Message Queuing Telemetry Transport)
(C) FTP
(D) Telnet

35. Which popular Python client library implements the MQTT protocol?

(A) pymqtt
(B) paho-mqtt
(C) iot-mqtt
(D) mqttclient

36. In MQTT terminology, what is the central message broker server that routes telemetry packets?

(A) Gateway
(B) Broker (e.g. Mosquitto)
(C) Router
(D) Relay

37. In MQTT, what are hierarchical strings (e.g. `sensors/room1/temperature`) used to filter messages called?

(A) URIs
(B) Topics
(C) Headers
(D) Keys

38. Which standard library module provides a lightweight, zero-configuration disk-based relational SQL database?

(A) psycopg2
(B) sqlite3
(C) mysqldb
(D) sqlalchemy

39. How do you execute an SQL command in Python using `sqlite3`?

(A) `cursor.run(sql)`
(B) `cursor.execute(sql)`
(C) `conn.query(sql)`
(D) `cursor.commit(sql)`

40. Why is `conn.commit()` required after executing INSERT or UPDATE statements in `sqlite3`?

(A) To compile SQL syntax
(B) To flush pending database transactions to physical disk storage
(C) To close connection
(D) To index tables

41. Which digital filter design function from `scipy.signal` calculates Butterworth filter transfer function poles and zeros?

(A) scipy.signal.cheby1()
(B) scipy.signal.butter()
(C) scipy.signal.ellip()
(D) scipy.signal.bessel()

42. What is the key characteristic of a Butterworth filter frequency response in its passband?

(A) Equiripple oscillations in passband
(B) Maximally flat magnitude response with zero ripple
(C) Steepest roll-off transition
(D) Linear phase response

43. Which filter type provides an equiripple passband with faster transition roll-off than Butterworth?

(A) Bessel
(B) Chebyshev Type I
(C) Gaussian
(D) Moving Average

44. Which filter type provides equiripple behavior in BOTH passband and stopband with the steepest possible transition band?

(A) Chebyshev Type II
(B) Elliptic (Cauer) filter
(C) Butterworth
(D) FIR boxcar

45. Which digital filter implementation function applies an IIR or FIR filter along a 1D signal with causal delay?

(A) scipy.signal.convolve()
(B) scipy.signal.lfilter()
(C) scipy.signal.sosfilt()
(D) scipy.signal.filter()

46. What is the primary advantage of `scipy.signal.filtfilt()` over `scipy.signal.lfilter()`?

(A) Runs twice as fast
(B) Performs forward-backward filtering to achieve exactly zero phase distortion (0° delay)
(C) Requires less RAM
(D) Only works on FIR

47. Which function designs linear-phase FIR filters using the windowed sinc method?

(A) scipy.signal.iirfilter()
(B) scipy.signal.firwin()
(C) scipy.signal.firls()
(D) scipy.signal.remez()

48. Which function computes the complex frequency response $H(e^{j\omega})$ of a digital filter from numerator and denominator coefficients?

(A) scipy.signal.step()
(B) scipy.signal.freqz()
(C) scipy.signal.bode()
(D) scipy.signal.impulse()

49. Why are Second-Order Sections (SOS) preferred over transfer function polynomial coefficients (b, a) for high-order IIR filters?

(A) Avoids severe numerical coefficient quantization errors and pole instability caused by finite floating-point precision
(B) Uses fewer CPU registers
(C) Eliminates need for sampling rate
(D) Converts IIR to FIR

50. Which function applies Second-Order Sections filtering with zero phase distortion?

(A) scipy.signal.sosfilt()
(B) scipy.signal.sosfiltfilt()
(C) scipy.signal.lfilter()
(D) scipy.signal.cascade()

51. In Amplitude Modulation (AM), how is a message signal $m(t)$ modulated onto carrier $A_c \cos(2\pi f_c t)$?

(A) $s(t) = [A_c + m(t)] \cos(2\pi f_c t)$
(B) $s(t) = A_c \cos(2\pi [f_c + m(t)] t)$
(C) $s(t) = m(t) \cdot A_c$
(D) $s(t) = A_c \cos(2\pi f_c t) / m(t)$

52. What is the modulation index μ in AM defined as when $m(t) = A_m \cos(2\pi f_m t)$?

(A) $\mu = A_m / A_c$
(B) $\mu = A_c / A_m$
(C) $\mu = f_m / f_c$
(D) $\mu = A_m \cdot A_c$

53. What occurs in envelope detection of an AM signal if modulation index $\mu > 1$?

(A) Signal power doubles
(B) Overmodulation envelope distortion (carrier phase reversal envelope inversion)
(C) Zero bandwidth
(D) Carrier suppression

54. How is the instantaneous frequency of a Frequency Modulated (FM) signal mathematically related to message $m(t)$?

(A) $f_i(t) = f_c + k_f m(t)$
(B) $f_i(t) = f_c \cdot m(t)$
(C) $f_i(t) = f_c / m(t)$
(D) $f_i(t) = f_c + k_f \int m(t) dt$

55. According to Carson's rule, what is the transmission bandwidth B_T of an FM signal?

(A) $B_T = 2 (\Delta f + f_m)$
(B) $B_T = 2 \Delta f$
(C) $B_T = 2 f_m$
(D) $B_T = \Delta f \cdot f_m$

56. Which function from `scipy.signal` performs Hilbert transform to extract the analytic signal and instantaneous envelope?

(A) scipy.signal.envelope()
(B) scipy.signal.hilbert()
(C) scipy.signal.analytic()
(D) scipy.signal.iq()

57. How do you extract the instantaneous amplitude envelope $A(t)$ from analytic signal $z(t)$ returned by `scipy.signal.hilbert`?

(A) `A = np.abs(z)`
(B) `A = np.real(z)`
(C) `A = np.imag(z)`
(D) `A = np.angle(z)`

58. How do you extract the instantaneous phase $\phi(t)$ from analytic signal $z(t)$?

(A) `phi = np.abs(z)`
(B) `phi = np.angle(z)` or `np.unwrap(np.angle(z))`
(C) `phi = np.real(z)`
(D) `phi = np.imag(z)`

59. What does `np.unwrap(phase)` do to a phase angle array?

(A) Wraps phase into $[-\pi, \pi]$
(B) Unwraps phase by adding multiples of $\pm 2\pi$ to eliminate discontinuous phase jumps
(C) Sets phase to zero
(D) Computes phase derivative

60. Which function from `scipy.optimize` fits non-linear parametric functions (e.g. diode Shockley equation) to experimental data?

(A) scipy.optimize.minimize()
(B) scipy.optimize.curve_fit()
(C) scipy.optimize.root()
(D) scipy.optimize.linprog()

61. What does `popt, pcov = scipy.optimize.curve_fit(f, xdata, ydata)` return?

(A) `popt` is optimal parameter values; `pcov` is estimated covariance matrix of parameters
(B) Fitted curve array
(C) Error residuals only
(D) R-squared value

62. How are standard errors of fitted parameters calculated from the covariance matrix `pcov`?

(A) `perr = np.sqrt(np.diag(pcov))`
(B) `perr = np.trace(pcov)`
(C) `perr = np.linalg.det(pcov)`
(D) `perr = np.sum(pcov)`

63. Which function from `scipy.optimize` finds local minima of unconstrained multivariable scalar functions?

(A) scipy.optimize.minimize()
(B) scipy.optimize.root()
(C) scipy.optimize.fmin()
(D) scipy.optimize.find_min()

64. What method in `scipy.integrate.solve_ivp` should be specified when solving stiff differential equations (e.g. fast switching transients)?

(A) 'RK45'
(B) 'Radau' or 'BDF'
(C) 'Euler'
(D) 'RK23'

65. What is a 'stiff' system of differential equations in circuit simulation?

(A) A system containing dynamic time constants spanning drastically different orders of magnitude (e.g. fast nanosecond gate switching alongside millisecond thermal decay)
(B) A system with zero resistance
(C) An uninvertible matrix
(D) A mechanical system

66. Which module provides threading capabilities in Python?

(A) process
(B) threading
(C) multiprocessing
(D) threadpool

67. What is the Global Interpreter Lock (GIL) in CPython?

(A) A mutex that prevents multiple native OS threads from executing Python bytecodes simultaneously in a single process
(B) A security firewall
(C) A file lock
(D) A hardware watchdog

68. Why does Python multithreading (`threading.Thread`) excel at I/O-bound tasks (like serial logging and network sockets) despite the GIL?

(A) CPython releases the GIL during blocking operating system system calls (e.g. reading from socket or COM port)
(B) It bypasses CPU completely
(C) It converts Python to C
(D) Threads don't use memory

69. Which module should be used instead of `threading` to achieve true parallel multi-core CPU execution for heavy numerical simulations?

(A) asyncio
(B) multiprocessing
(C) concurrent.futures.ThreadPoolExecutor
(D) subprocess only

70. What thread-safe data structure from the `queue` module is standard for passing telemetry data between worker threads?

(A) queue.Queue
(B) list
(C) collections.deque without lock
(D) dict

71. Which built-in module provides asynchronous event-loop concurrency (`async`/`await`) for high-scale network services?

(A) concurrency
(B) asyncio
(C) twisted
(D) coroutine

72. What does `await asyncio.sleep(1.0)` do in an async coroutine?

(A) Freezes the entire operating system thread
(B) Yields control back to the event loop to run other scheduled coroutines while waiting 1 second
(C) Terminates the program
(D) Runs a background thread

73. Which function from `time` module provides monotonic, high-precision timing for measuring DSP algorithm execution intervals?

(A) time.time()
(B) time.perf_counter()
(C) time.process_time()
(D) time.clock()

74. Why is `time.perf_counter()` preferred over `time.time()` for benchmarking algorithms?

(A) `time.perf_counter()` uses monotonic hardware clocks that cannot be stepped backward by system NTP clock synchronizations
(B) It returns integers
(C) It measures GPU time
(D) It runs faster

75. Which built-in module provides deterministic profiling of Python programs to pinpoint execution bottlenecks?

(A) timeit
(B) cProfile
(C) profiler
(D) trace

76. How is a continuous-time linear state-space model $\dot{x} = A x + B u$ discretized into $x_{k+1} = A_d x_k + B_d u_k$ using Zero-Order Hold (ZOH)?

(A) $A_d = e^{A T_s}, \quad B_d = \left(\int_0^{T_s} e^{A \tau} d\tau \right) B$
(B) $A_d = I + A T_s, \quad B_d = B T_s$
(C) $A_d = (I - A T_s)^{-1}$
(D) $A_d = A / T_s$

77. Which function from `scipy.signal` converts continuous-time LTI system state-space models to discrete-time models?

(A) scipy.signal.cont2discrete()
(B) scipy.signal.discretize()
(C) scipy.signal.sample_system()
(D) scipy.signal.zoh()

78. What transformation method does `scipy.signal.cont2discrete(..., method='bilinear')` employ?

(A) Euler forward difference
(B) Tustin bilinear conformal mapping ($s = \frac{2}{T_s} \frac{z - 1}{z + 1}$)
(C) Runge-Kutta 4th order
(D) Impulse invariance

79. In Software-Defined Radio (SDR), what do In-phase (I) and Quadrature (Q) baseband components represent?

(A) $I(t) = \text{Re}\{s_b(t)\}$ and $Q(t) = \text{Im}\{s_b(t)\}$ representing Cartesian decomposition of complex envelope $s_b(t) = I(t) + j Q(t)$
(B) Input and Output voltage
(C) Integer and Quantitative data
(D) Frequency and Period

80. How is complex baseband signal $s_b(t) = I(t) + j Q(t)$ upconverted to RF passband carrier frequency f_c?

(A) $s_{RF}(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t)$
(B) $s_{RF}(t) = I(t) \sin(2\pi f_c t) + Q(t) \cos(2\pi f_c t)$
(C) $s_{RF}(t) = I(t) \cdot Q(t) \cos(2\pi f_c t)$
(D) $s_{RF}(t) = (I + Q) \cos(2\pi f_c t)$

81. In digital Binary Phase Shift Keying (BPSK), what are the nominal constellation coordinates in the complex I/Q plane?

(A) $\{+1 + j0, -1 + j0\}$
(B) $\{0 + j1, 0 - j1\}$
(C) $\{1 + j1, -1 - j1\}$
(D) $\{+1, 0\}$

82. What is the theoretical Bit Error Rate (BER) of coherent BPSK modulation over an Additive White Gaussian Noise (AWGN) channel?

(A) $P_b = Q\left(\sqrt{\frac{2 E_b}{N_0}} \right) = \frac{1}{2} \text{erfc}\left(\sqrt{\frac{E_b}{N_0}} \right)$
(B) $P_b = e^{-E_b/N_0}$
(C) $P_b = \frac{1}{2} \left(1 - \frac{E_b}{N_0} \right)$
(D) $P_b = Q\left(\frac{E_b}{N_0} \right)$

83. Which function from `scipy.special` computes the complementary error function $\text{erfc}(x)$ for BER evaluation?

(A) scipy.special.erfc()
(B) scipy.special.error()
(C) scipy.special.q_func()
(D) scipy.special.ber()

84. In Quadrature Amplitude Modulation (16-QAM), how many bits are transmitted per modulation symbol?

(A) 2 bits
(B) 4 bits (since $2^4 = 16$)
(C) 8 bits
(D) 16 bits

85. What is a Matched Filter in digital receiver design, and what fundamental metric does it maximize at the sampling instant?

(A) It maximizes the output Signal-to-Noise Ratio (SNR) in the presence of stochastic AWGN noise
(B) It minimizes bandwidth
(C) It maximizes carrier frequency
(D) It eliminates harmonic distortion

86. In time-domain, how is the impulse response $h(t)$ of a matched filter related to the transmitted pulse shape $s(t)$?

(A) $h(t) = s^*(T - t)$ (time-reversed and complex conjugated replica)
(B) $h(t) = s(t)$
(C) $h(t) = 1 / s(t)$
(D) $h(t) = d s(t) / dt$

87. What standard pulse shaping filter is used in wireless modems to achieve zero Intersymbol Interference (ISI) while satisfying Nyquist criterion?

(A) Root-Raised Cosine (RRC) filter
(B) Butterworth filter
(C) Rectangular boxcar filter
(D) Chebyshev filter

88. What does the excess bandwidth parameter α (roll-off factor, $0 \le \alpha \le 1$) control in a Raised Cosine filter?

(A) The trade-off between occupied frequency bandwidth $B = \frac{R_s}{2}(1 + \alpha)$ and time-domain pulse ringing decay rate
(B) The filter DC gain
(C) The ADC bit depth
(D) The carrier phase

89. Which standard instrument protocol defines human-readable text commands (e.g. `*IDN?`, `MEAS:VOLT:DC?`) for programmable laboratory equipment?

(A) SCPI (Standard Commands for Programmable Instruments)
(B) MODBUS
(C) CANbus
(D) I2C

90. Which Python library interfaces with VISA (Virtual Instrument Software Architecture) test equipment via GPIB, USB, and Ethernet?

(A) PyVISA
(B) pyserial
(C) scpi-py
(D) labview-python

91. How do you query an instrument identity string using PyVISA?

(A) `inst.query('*IDN?')`
(B) `inst.ask('ID')`
(C) `inst.get_name()`
(D) `inst.read('*IDN?')`

92. What is the purpose of the CRC-16 or CRC-32 (Cyclic Redundancy Check) algorithm in serial telemetry frames?

(A) Detecting single-bit, double-bit, and burst transmission errors caused by noise on communication links
(B) Data encryption
(C) Data compression
(D) Packet routing

93. Which built-in module computes CRC-32 checksums in Python at high speeds?

(A) hashlib
(B) binascii (`binascii.crc32`)
(C) zlib (`zlib.crc32`)
(D) Both binascii.crc32 and zlib.crc32

94. What does `struct.pack('>HHf', 0xAA55, 100, 3.14159)` do in binary telemetry serialization?

(A) Packs integers and a float into big-endian contiguous binary bytes according to the format string
(B) Parses JSON string
(C) Compresses file
(D) Encodes UTF-8 text

95. What format character in `struct` module represents big-endian network byte ordering?

(A) '>' or '!'
(B) '<'
(C) '='
(D) '@'

96. What format character in `struct` represents a 32-bit standard IEEE 754 floating-point number?

(A) 'f'
(B) 'd'
(C) 'i'
(D) 'q'

97. Which Python library enables real-time low-latency audio capture and playback through PortAudio?

(A) wave
(B) PyAudio
(C) soundfile
(D) audiolib

98. In real-time digital signal processing, what is 'buffer underrun' or 'buffer overrun' (dropouts)?

(A) When the audio/DAC buffer runs out of processed samples (underrun) or ADC fills buffer before CPU reads it (overrun), causing audible clicks/glitches
(B) Hardware power failure
(C) Memory leak
(D) Floating-point overflow

99. What technique divides computational work among a pool of worker processes to bypass the GIL for parameter sweeps?

(A) `concurrent.futures.ProcessPoolExecutor` or `multiprocessing.Pool`
(B) `threading.Thread`
(C) `asyncio.gather`
(D) `generator`

100. What is Hardware-in-the-Loop (HIL) simulation in embedded electronics engineering?

(A) A testing technique where physical microcontroller hardware or electronic control units (ECUs) are connected directly to real-time software simulations of plants/circuits
(B) Running Python inside FPGA
(C) Simulating CPU silicon gates
(D) Compiling Python to C

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Formulate the mathematical model of a series RLC resonant circuit and solve for its transient step response using `scipy.integrate.solve_ivp`.

Q2. Design an active 4th-order low-pass Butterworth IIR filter in SciPy and evaluate its magnitude and phase frequency response using `freqz`.

Q3. Compare causal IIR filtering (`lfilter`) versus zero-phase non-causal forward-backward filtering (`filtfilt`) with mathematical derivations.

Q4. Implement an Amplitude Modulation (AM) and Envelope Demodulation simulation in Python with Additive White Gaussian Noise (AWGN).

Q5. Write an end-to-end Python script using `pySerial` to configure an RF synthesizer instrument, send SCPI commands, and parse responses.

Q6. Implement a robust multi-threaded TCP telemetry server in Python that accepts multiple wireless sensor clients and logs data.

Q7. Formulate and implement the classical 4th-Order Runge-Kutta (RK4) numerical ODE solver from scratch in Python.

Q8. Design a Monte Carlo simulation in Python to evaluate the Bit Error Rate (BER) of BPSK over an AWGN channel against theoretical $Q(\sqrt{2 E_b/N_0})$.

Q9. Implement a real-time sensor telemetry logger using Python `sqlite3` with indexed timestamp schema and statistical aggregation queries.

Q10. Demonstrate high-precision pitch and dominant frequency detection in audio signals using FFT with parabolic interpolation.

Q11. Explain the implementation of an MQTT publisher and subscriber in Python using `paho-mqtt` for an environmental monitoring system.

Q12. Detail the design and implementation of an asynchronous TCP sensor hub in Python using `asyncio`.

Q13. Demonstrate diode I-V Shockley equation parameter extraction using non-linear least squares curve fitting with `scipy.optimize.curve_fit`.

Q14. Implement a custom binary packet parser using Python's `struct` module for embedded CANbus or UART protocols with CRC checks.

Q15. Explain the architecture of a Software-Defined Radio (SDR) Costas Loop for carrier phase synchronization in BPSK.

Q16. Implement a vectorized Root-Raised Cosine (RRC) pulse shaping filter generator in NumPy for Nyquist signaling.

Q17. Demonstrate continuous-time state-space simulation of a closed-loop PID DC motor angular speed controller.

Q18. Explain the principles of Software-Defined Radio (SDR) Quadrature Amplitude Modulation (16-QAM) constellation mapping and demapping.

Q19. Demonstrate microsecond algorithm execution profiling using `cProfile` and memory profiling in scientific Python pipelines.

Q20. Implement an automated Hardware-in-the-Loop (HIL) test suite in Python executing automated test vectors and asserting tolerance bands.

Q21. Implement an automated discrete Phase-Locked Loop (PLL) in Python for carrier frequency tracking.

Q22. Write a Python script for automated multi-channel oscilloscope waveform acquisition via PyVISA and USBTMC.

Q23. Demonstrate circular buffer design using Python `multiprocessing.shared_memory` for multi-process IPC data streaming.

Q24. Implement an adaptive Wiener filter in Python for optimal signal restoration in colored noise environments.

Q25. Explain the implementation of a Discrete Wavelet Transform (DWT) multi-resolution decomposition using `PyWavelets`.

Q26. Implement a custom PID speed controller simulation with anti-windup clamping in Python.

Q27. Design an FIR Hilbert Transformer filter in Python for creating single-sideband (SSB) modulated communications signals.

Q28. Implement an automated Peak-to-Average Power Ratio (PAPR) reduction simulation using Selected Mapping (SLM) for OFDM.

Q29. Explain the architecture and design of an automated SCPI command parser and emulator in Python.

Q30. Implement an automated test reporting framework combining DSP simulations, statistics, and Matplotlib figure export.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[B]	**Q21:**	[A]	**Q41:**	[B]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[B]	**Q22:**	[B]	**Q42:**	[B]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[B]	**Q23:**	[B]	**Q43:**	[B]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[B]	**Q24:**	[B]	**Q44:**	[B]	**Q64:**	[B]	**Q84:**	[B]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[B]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[B]	**Q26:**	[B]	**Q46:**	[B]	**Q66:**	[B]	**Q86:**	[A]
Q7:	[B]	**Q27:**	[B]	**Q47:**	[B]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[B]	**Q48:**	[B]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[B]	**Q29:**	[B]	**Q49:**	[A]	**Q69:**	[B]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[B]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[B]	**Q31:**	[B]	**Q51:**	[A]	**Q71:**	[B]	**Q91:**	[A]
Q12:	[B]	**Q32:**	[B]	**Q52:**	[A]	**Q72:**	[B]	**Q92:**	[A]
Q13:	[B]	**Q33:**	[B]	**Q53:**	[B]	**Q73:**	[B]	**Q93:**	[D]
Q14:	[B]	**Q34:**	[B]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[B]	**Q35:**	[B]	**Q55:**	[A]	**Q75:**	[B]	**Q95:**	[A]
Q16:	[B]	**Q36:**	[B]	**Q56:**	[B]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[B]	**Q37:**	[B]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[B]
Q18:	[A]	**Q38:**	[B]	**Q58:**	[B]	**Q78:**	[B]	**Q98:**	[A]
Q19:	[B]	**Q39:**	[B]	**Q59:**	[B]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[B]	**Q60:**	[B]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

Python Implementation using Second-Order Sections (SOS):
```python
import numpy as np
from scipy import signal

fs = 48000.0 # 48 kHz sampling rate
fc = 4000.0 # 4 kHz cutoff frequency
order = 4

# Design filter returning Second-Order Sections (SOS)
sos = signal.butter(order, fc, btype='lowpass', fs=fs, output='sos')

# Compute complex frequency response
w, h = signal.sosfreqz(sos, worN=2048, fs=fs)
mag_db = 20 * np.log10(np.maximum(np.abs(h), 1e-6))
phase_deg = np.unwrap(np.angle(h)) * (180 / np.pi)
```

Evaluation: At $f = 4$ kHz, the gain is precisely -3.01 dB, and in the stopband ($f > 8$ kHz), the attenuation rolls off at $-20 \times N = -80$ dB/decade.

```python
import numpy as np
from scipy import signal

fs = 200000.0 # 200 kHz sampling rate
t = np.linspace(0, 0.01, int(0.01 * fs), endpoint=False) # 10 ms

# Message: 1 kHz tone; Carrier: 20 kHz tone
fm, fc = 1000.0, 20000.0
m_t = np.cos(2 * np.pi * fm * t)
mu = 0.8 # Modulation index 80%
Ac = 1.0

# AM Double-Sideband with Carrier (DSB-WC)
s_tx = Ac * (1 + mu * m_t) * np.cos(2 * np.pi * fc * t)

# Channel AWGN: Add noise for 15 dB SNR
p_sig = np.mean(s_tx**2)
p_noise = p_sig / (10**(15.0 / 10.0))
noise = np.random.normal(0, np.sqrt(p_noise), len(t))
s_rx = s_tx + noise

# Demodulation via Analytic Signal (Hilbert Transform)
analytic_rx = signal.hilbert(s_rx)
env_rx = np.abs(analytic_rx)

# Remove DC carrier offset to reconstruct original message
m_demod = (env_rx - np.mean(env_rx)) / (Ac * mu)
```

>> Solution to Question 5:

```
# Format command with standard carriage-return/newline
full_cmd = (cmd.strip() + '\r\n').encode('ascii')
self.ser.write(full_cmd)
# Read response line
raw_resp = self.ser.readline()
return raw_resp.decode('ascii').strip()

def set_rf_frequency(self, freq_mhz: float):
resp = self.query(f':FREQ:CW {freq_mhz} MHz')
status = self.query(':SYST:ERR?')
return status

def close(self):
self.ser.close()
```

Solution to Question 6:

>>> Problem Statement: Implement a robust multi-threaded TCP telemetry server in Python that accepts multiple wireless sensor clients and logs data.

```
  client, addr = self.server_sock.accept()
  t = threading.Thread(target=self.handle_client, args=(client, addr), daemon=True)
  t.start()
```

```
ber_sim = []
ber_theory = []

# 1. Generate random binary bits {0, 1}
bits = np.random.randint(0, 2, num_bits)
# 2. BPSK mapping: 0 -> +1 V, 1 -> -1 V
s = 1.0 - 2.0 * bits

for snr_db in snr_db_range:
snr_linear = 10.0 ** (snr_db / 10.0)
# Noise variance: sigma^2 = Eb / (2 * SNR)
sigma = np.sqrt(1.0 / (2.0 * snr_linear))
noise = np.random.normal(0, sigma, num_bits)
r = s + noise

# Optimal threshold decision at 0V
detected_bits = (r < 0.0).astype(int)
errors = np.sum(bits != detected_bits)
ber_sim.append(errors / num_bits)

# Theoretical Q(sqrt(2 * Eb/N0))
theory = 0.5 * erfc(np.sqrt(snr_linear))
ber_theory.append(theory)

return np.array(ber_sim), np.array(ber_theory)
```

Solution to Question 9:

>>> Solution to Question 9: Implement a real-time sensor telemetry logger using Python `sqlite3` with indexed timestamp schema and statistical aggregation queries

```
# B-Tree index on timestamp for fast time-range queries
self.conn.execute('CREATE INDEX IF NOT EXISTS idx_time ON sensor_logs(timestamp)')

def insert_batch(self, records):
with self.conn:
self.conn.executemany(
'INSERT INTO sensor_logs (timestamp, node_id, voltage, rssi_dbm) VALUES (?, ?, ?, ?)',
records
)

def query_hourly_stats(self, node_id):
cursor = self.conn.cursor()
cursor.execute('''
SELECT AVG(voltage), MIN(voltage), MAX(voltage), COUNT(*)
FROM sensor_logs WHERE node_id = ? AND timestamp >= ?
''', (node_id, time.time() - 3600))
return cursor.fetchone()
```

Solution to Question 10:

```


```
>>> Solution to Question 10:
```
```
```
```

>>Solution Question 12: The implementation of an IoT application for environmental monitoring system.

```
finally:
writer.close()
await writer.wait_closed()

async def main():
server = await asyncio.start_server(handle_sensor_client, '0.0.0.0', 8888)
async with server:
await server.serve_forever()
```

```
HEADER_MAGIC = 0xAA55

def pack_telemetry(msg_id: int, timestamp: int, v1: float, v2: float) -> bytes:
# Pack preliminary packet without CRC
prelim = struct.pack('>HB I f f', HEADER_MAGIC, msg_id, timestamp, v1, v2)
crc = zlib.crc32(prelim)
return struct.pack('>HB I f f I', HEADER_MAGIC, msg_id, timestamp, v1, v2, crc)

def unpack_telemetry(raw_bytes: bytes):
if len(raw_bytes) != struct.calcsize(PACKET_FMT):
raise ValueError('Invalid packet length')
magic, msg_id, t_stamp, v1, v2, rx_crc = struct.unpack(PACKET_FMT, raw_bytes)
if magic != HEADER_MAGIC:
raise ValueError('Invalid magic byte header')
# Verify CRC
calc_crc = zlib.crc32(raw_bytes[:-4])
if rx_crc != calc_crc:
raise ValueError(f'CRC mismatch: expected {calc_crc}, got {rx_crc}')
return {'id': msg_id, 'time': t_stamp, 'v1': v1, 'v2': v2}
```

>>Solution Question 16: [illegible] **Solution Question 16:** [illegible]

```
h = np.zeros(len(t))

for i, ti in enumerate(t):
if ti == 0.0:
h[i] = 1.0 - alpha + (4 * alpha / np.pi)
elif abs(ti) == Ts / (4 * alpha):
h[i] = (alpha / np.sqrt(2)) * (
(1 + 2 / np.pi) * np.sin(np.pi / (4 * alpha)) +
(1 - 2 / np.pi) * np.cos(np.pi / (4 * alpha))
)
else:
num = np.sin(np.pi * ti * (1 - alpha) / Ts) + \
(4 * alpha * ti / Ts) * np.cos(np.pi * ti * (1 + alpha) / Ts)
den = (np.pi * ti / Ts) * (1 - (4 * alpha * ti / Ts)**2)
h[i] = num / den

return h / np.sqrt(np.sum(h**2)) # Unit energy normalization
```

[Detailed Step-by-Step Answer]:
16-QAM Modulator and Slicer Mechanics:

16-QAM transmits 4 bits per symbol: 2 bits for In-phase (I) and 2 bits for Quadrature (Q).

Gray Mapping:
Constellation alphabet: $\{-3d, -d, +d, +3d\}$ along each axis.
```python
import numpy as np

def qam16_modulate(bits):
# Reshape bits into (N, 4)
symbols = bits.reshape(-1, 4)
lut = { (0,0): -3, (0,1): -1, (1,1): +1, (1,0): +3 }
I = np.array([lut[(b[0], b[1])] for b in symbols])
Q = np.array([lut[(b[2], b[3])] for b in symbols])
return I + 1j * Q

def qam16_demap(complex_symbols):
# Slicing at decision boundaries: -2, 0, +2
def slice_axis(val):
if val < -2: return (0, 0)
elif val < 0: return (0, 1)
elif val < 2: return (1, 1)
else: return (1, 0)
out_bits = []
for s in complex_symbols:
out_bits.extend(slice_axis(s.real) + slice_axis(s.imag))
return np.array(out_bits)
```

>> Solution to Question 19: Demonstrate microsecond algorithm execution profiling using `cProfile` and memory profiling in scientific Python pipelines.

return result
```

Analysis reveals whether time is consumed by C-level BLAS calls, unnecessary Python function loop wrappers, or memory reallocations, guiding targeted optimization.

---

**>>P Solution Question 20.** Implement an automated Hardware-in-the-Loop (HIL) test framework for a real-time embedded controller. Your solution must use Python to generate randomized test vectors and assert tolerance bar
```

```
    omega_c = 2 * np.pi * f_carrier / fs

    for n in range(N):
        # Quadrature mixer phase detector
        q_nco = -np.sin(phi_nco)
        phase_error = rx_signal[n] * q_nco

        # PI Loop Filter
        integrator += Ki * phase_error
        loop_out = Kp * phase_error + integrator

        # NCO phase accumulator
        phi_nco += omega_c + loop_out
        phase_est[n] = phi_nco

    return phase_est
```

Solution to Question 22:

>>> Here is a Python script for automated multi-channel oscilloscope waveform acquisition via PyVISA and USBTMC.

Discrete Wavelet Transform Multi-Resolution Analysis:

```python
import pywt
import numpy as np

def wavelet_denoise(signal_data, wavelet='db4', level=4, threshold=0.1):
# Multi-level DWT decomposition
coeffs = pywt.wavedec(signal_data, wavelet, level=level)

# Soft thresholding on high-frequency detail coefficients
coeffs_thresh = [coeffs[0]] # Keep approximation coefficients
for c in coeffs[1:]:
c_t = pywt.threshold(c, threshold, mode='soft')
coeffs_thresh.append(c_t)

# Inverse DWT reconstruction
clean_signal = pywt.waverec(coeffs_thresh, wavelet)
return clean_signal
```

Solution to Question 26:

>> **Solution to Question 26:** Implement a custom PID speed controller simulation with anti-windup clamping in Python.

```
    if papr_db < best_papr:
        best_papr = papr_db
        best_waveform = x_time

    return best_papr, best_waveform
```

Solution Question 30: Implement the architecture and test vector framework for a 3GPP-compliant DSP simulation environment using Python and Matplotlib figure export.

```
  s21 = test_results['s21_db']

  fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 7))
  ax1.plot(freqs, s11, 'r-', linewidth=1.8, label='Return Loss (S11)')
  ax1.axhline(-10, color='k', linestyle='--', label='-10 dB Spec')
  ax1.set_ylabel('S11 (dB)')
  ax1.grid(True)
  ax1.legend()

  ax2.plot(freqs, s21, 'b-', linewidth=1.8, label='Insertion Gain (S21)')
  ax2.set_xlabel('Frequency (GHz)')
  ax2.set_ylabel('S21 (dB)')
  ax2.grid(True)
  ax2.legend()

  plt.suptitle('RF Front-End Automated Qualification Summary', fontsize=13, weight='bold')
  plt.tight_layout()
  plt.savefig(output_img, dpi=300)
  plt.close()
  print(f'[+] Report generated and saved to {output_img}')
```
